

Live Web Application:
<https://harshitasodiya1.github.io/financeTraker/>

Core Architecture: Privacy-First, 100% Client-Side

GitHub Source Code:
<https://github.com/Harshitasodiya1/financeTraker>

Persistence: Dual Storage (`localStorage` + `IndexedDB`)

1. PROJECT SYNOPSIS (EXECUTIVE SUMMARY)

1.1 Problem Statement

Existing personal expense tracking applications typically suffer from three critical drawbacks: (1) intrusive sign-up flows that demand bank credentials, (2) storage of personal financial records on third-party cloud servers creating privacy vulnerabilities, and (3) cluttered user interfaces filled with advertisements and overwhelming menus. Traditional spreadsheets solve the privacy issue but lack responsiveness on mobile devices and provide no automated real-time visual feedback.

1.2 Solution & Core Objectives

FinFlow solves this by offering a zero-clutter, Apple-inspired client-side web application. It runs 100% locally in the user's browser, meaning sensitive financial data never leaves their device. It features full support for the Indian numbering system (`₹1,00,000` Lakhs/Crores), dynamic charts, category budget controls with automated overrun warnings, multi-filter search, and instant data portability via CSV export and JSON backup.

1.3 Technology Stack

Layer / Tool	Technologies	Rationale / Why Chosen?
Frontend Core	HTML5 (Semantic) & Vanilla JS (ES6+)	Zero build pipeline overhead, instant load speed, near-zero bundle size, and solid proof of deep core JavaScript & DOM mastery.
Styling & UI	Tailwind CSS & Custom Design System	Rapid utility styling, Apple-style segmented tabs, fluid responsive grid layout, and seamless Dark/Light mode switching.
Visual Analytics	Chart.js 4.x (HTML5 Canvas)	Lightweight, animated, high-performance canvas rendering for Donut, Bar, and Line charts with smooth real-time re-rendering.
Persistence	<code>localStorage</code> + <code>IndexedDB</code>	Dual-tier offline persistence. Zero backend needed; data remains permanently preserved across browser sessions and device restarts.
Hosting & CI/CD	Git & GitHub Pages	Zero-cost global edge hosting with Git commit-triggered automatic deployment.

2. TECHNICAL INTERVIEW QUESTIONS & ANSWERS

Q1. Can you walk me through your project? What does it do?

"FinFlow is a client-side personal finance tracker engineered with a privacy-first philosophy and an Apple/Fi Money minimalist design system. It addresses user privacy concerns by storing all transaction and budget data entirely inside the user's browser using `localStorage` and `IndexedDB`. Users can track income and expenses across 8+ categories, view visual analytics via `Chart.js`, set monthly budget spending thresholds with warning alerts, filter transactions in real time, and export their data to CSV or JSON backup."

Q2. Why did you build this in Vanilla JavaScript instead of React or Vue?

"I chose Vanilla JavaScript deliberately to demonstrate fundamental mastery of modern ES6+ JavaScript—including manual DOM manipulation, custom state management, event delegation, and browser storage APIs. It also eliminates build steps, avoids Virtual DOM overhead, and ensures lightning-fast initial load times."

Q3. How is data persistence handled? What happens if the page is refreshed?

"The application uses a **Dual-Tier Persistence Architecture**. When a transaction is recorded, it is converted to JSON and written synchronously to `localStorage` for rapid reads during page hydration, and simultaneously mirrored into an `IndexedDB` database for resilience. When the page reloads, data is immediately restored from storage. Users also have one-click JSON backup and CSV export options."

Q4. How does Indian Rupee (INR) formatting differ, and how did you implement it?

"The Western numbering system groups digits in threes (`1,000,000`). In contrast, the Indian system groups the first three digits and subsequently groups in pairs of two (`₹1,00,000` for 1 Lakh, `₹1,00,00,000` for 1 Crore). I implemented this using the browser's native `Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR' })` API, which guarantees standard-compliant Indian Rupee formatting without fragile regex."

Q5. How does the multi-criteria search and filter engine function?

"The filter operates on an active state object containing `searchQuery`, `type`, `category`, and `sortBy`. Whenever the user inputs text or adjusts a dropdown, the full transaction dataset passes through functional `Array.prototype.filter()` and `Array.prototype.sort()` methods. Only the matching subset is re-rendered to the DOM, and summary totals (record count and Net Flow) update dynamically."

Q6. How are the charts kept synchronized with data additions and deletions?

"I maintain three global `Chart.js` instances (Donut, Bar, and Line). When a transaction is added, edited, or deleted, an `updateCharts()` routine executes. It aggregates category totals using `Array.prototype.reduce()`, updates the dataset arrays (`chart.data.datasets[0].data`), and calls `chart.update()` to trigger smooth animated visual transitions."

Q7. How do the category Budget Limits and Overspend Alerts work?

"Users define monthly spending caps per category. The app computes current-month cumulative spending for each category:

- If spending is < 80%, the progress bar displays green (`emerald-500`).
- If between 80%–99%, it shifts to a warning amber (`amber-500`).
- If spending reaches or exceeds 100%, the bar turns red (`rose-500`) and displays an *'Exceeded by ₹X'* alert badge."

Q8. How does date grouping ('Today', 'Yesterday', 'Older') work?

"Each transaction timestamp is compared mathematically against `new Date()` . If the calendar date matches today, it groups under **Today**; if it matches yesterday, it groups under **Yesterday**; otherwise, it formats under **Older** with a formatted date string. Relative timestamps ('2 hours ago', 'Just now') are computed using minute/hour deltas."

Q9. How did you implement Dark Mode without theme flickering (FOUC)?

"Tailwind is configured for class-based dark mode (`<html class='dark'>`). An inline script reads `localStorage.getItem('theme')` before the DOM paints. If dark mode was previously selected, the `dark` class is added immediately to prevent screen flickering (Flash of Unstyled Content). If no preference is stored, it defaults to the OS setting via `window.matchMedia('(prefers-color-scheme: dark)')` ."

Q10. How is client-side CSV Data Export implemented without a server?

"The export runs purely on the client side:

1. Transactions are serialized into comma-separated text strings.
2. A binary `Blob` is created with MIME type `text/csv;charset=utf-8;` .
3. An in-memory object URL is generated using `URL.createObjectURL(blob)` .
4. A temporary anchor tag is created, triggered with `.click()` , and the object URL is revoked immediately."

Q11. What real deployment or technical challenge did you face and resolve?

"During GitHub Pages deployment, the HTML structure rendered but all button clicks and charts failed because `js/app.js` was returning an HTTP 404 error due to a path hierarchy mismatch in the root repository. I inspected the browser DevTools Network tab, identified the missing file path, placed `app.js` into the root `/js/` directory, and pushed to GitHub. Within 60 seconds, GitHub Pages served the script with an HTTP 200 OK status, restoring full functionality."

Q12. What features would you implement next if given more time?

"Three key enhancements:

1. **PWA (Progressive Web App)**: Add a service worker and manifest to make FinFlow installable on Android and iOS devices with offline capability.
2. **Receipt OCR**: Client-side bill scanning using Tesseract.js to automatically parse receipt amounts and dates.
3. **Recurring Payments**: Auto-reminders and automated logging for monthly subscriptions (Netflix, SIP, Rent)."